Project Name
AI Comic Story Creator using Gemini Model
Topic: Save Environment
Characters: Arun, Priya
Language: Tamil
Scenes: 6
Python — main programming
Gemini API — AI story generation
Streamlit — website/interface
Pillow — image handling
PDF — final comic export
AI_Comic_Story_Creator/
│
├── app.py
├── gemini_api.py
├── comic_generator.py
├── requirements.txt
└── output/

```
import streamlit as st

st.title("🎨 AI Comic Story Creator")

topic = st.text_input("Enter your story topic")
characters = st.text_input("Enter character names")

language = st.selectbox(
    "Select Language",
    ["Tamil", "English"]
)

scenes = st.number_input(
    "Number of scenes",
    min_value=2,
    max_value=10,
    value=6
)

if st.button("Generate Comic"):
    st.success("Comic generation started!")
from google import genai

client = genai.Client(api_key="YOUR_GEMINI_API_KEY")


def generate_comic_story(topic, characters, language, scenes):

    prompt = f"""
```

```
    Create an educational comic story.

    Topic: {topic}
    Characters: {characters}
    Language: {language}
    Number of scenes: {scenes}

    For every scene provide:
    1. Scene description
    2. Character dialogue
    3. Narration
    4. Image generation prompt

    Make the story simple, interesting and suitable for students.
    """

    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=prompt
    )

    return response.text
import streamlit as st
from gemini_api import generate_comic_story

st.title("🎨 AI Comic Story Creator")

topic = st.text_input("Enter your story topic")
characters = st.text_input("Enter character names")

language = st.selectbox(
    "Select Language",
    ["Tamil", "English"]
)

scenes = st.number_input(
    "Number of scenes",
    min_value=2,
    max_value=10,
    value=6
)

if st.button("Generate Comic"):

    if topic and characters:

        with st.spinner("Creating your comic story..."):
```

```
            story = generate_comic_story(
                topic,
                characters,
                language,
                scenes
            )

        st.subheader("📖 Generated Comic Story")
        st.write(story)

    else:
        st.warning("Please enter topic and character names.")
```

```
Topic: Save Environment
Characters: Arun, Priya
Language: Tamil
Scenes: 3
SCENE 1
Scene description:
Arun sees plastic waste near his school.

DIALOGUE:
Arun:
"நம்ம பள்ளி அருகில் இவ்வளவு குப்பை இருக்கிறதே!"

NARRATION:
Arun decides to do something about it.

IMAGE PROMPT:
Two Indian school students seeing plastic waste
near their school, colorful educational comic style.
```

```
from PIL import Image, ImageDraw, ImageFont


def create_comic_panel(scene_text, output_file):
    width = 1000
    height = 700

    image = Image.new("RGB", (width, height), "white")
    draw = ImageDraw.Draw(image)

    # Comic panel border
    draw.rectangle(
        (20, 20, width - 20, height - 20),
        outline="black",
        width=5
    )

    # Scene text
```

```
    draw.text(
        (50, 50),
        scene_text,
        fill="black"
    )

    image.save(output_file)

    return output_file
if story:

    st.subheader("🖼️ Comic Panels")

    panel_file = create_comic_panel(
        story,
        "output/comic_panel.png"
    )

    st.image(
        panel_file,
        caption="Generated Comic Panel"
    )
User Topic
    ↓
Gemini Model
    ↓
Comic Story
    ↓
Scene + Dialogue
    ↓
Image Prompt
    ↓
Comic Panel
    ↓
Final Comic
from google import genai

client = genai.Client(api_key="YOUR_GEMINI_API_KEY")


def generate_comic_image(image_prompt, output_file):

    response = client.models.generate_content(
        model="gemini-2.5-flash-image",
        contents=image_prompt
    )

    for part in response.parts:
```

```
        if part.inline_data is not None:
            image = part.as_image()
            image.save(output_file)
            return output_file

    return None
if st.button("Generate Comic Image"):

    image_prompt = """
    A colorful educational comic panel showing two
    Indian school students cleaning plastic waste
    near their school. Friendly cartoon style,
    bright colors, clear expressions.
    """

    image_file = generate_comic_image(
        image_prompt,
        "output/scene1.png"
    )

    if image_file:
        st.image(
            image_file,
            caption="Scene 1"
        )
    else:
        st.error("Image generation failed.")
Topic
 ↓
Gemini
 ↓
Story
 ↓
Scenes
 ↓
Dialogue + Image Prompt
 ↓
AI Image Generation
 ↓
Scene 1 Image
Scene 2 Image
Scene 3 Image
...
 ↓
Complete Comic 📖
Scene 1 → School + students + waste 🏫
Scene 2 → Students discussing recycling ♻️
Scene 3 → Students cleaning the area 🌱
```

Scene 4 → Students planting trees 🌳

```
from reportlab.pdfgen import canvas
from reportlab.lib.pagesizes import A4
from reportlab.lib.utils import ImageReader


def create_comic_pdf(panels, output_file):

    width, height = A4

    pdf = canvas.Canvas(output_file, pagesize=A4)

    for panel in panels:

        # Title
        pdf.setFont("Helvetica-Bold", 18)
        pdf.drawString(40, height - 50, panel["title"])

        # Comic image
        image = ImageReader(panel["image"])

        pdf.drawImage(
            image,
            40,
            300,
            width=515,
            height=350,
            preserveAspectRatio=True
        )

        # Narration
        pdf.setFont("Helvetica", 12)
        pdf.drawString(
            40,
            250,
            "Narration: " + panel["narration"]
        )

        # Dialogue
        pdf.drawString(
            40,
            220,
            "Dialogue: " + panel["dialogue"]
        )

        pdf.showPage()

    pdf.save()
```

```
    return output_file
panels = [
    {
        "title": "Scene 1",
        "image": "output/scene1.png",
        "narration": "Arun sees plastic waste near his school.",
        "dialogue": "Arun: We should keep our school clean!"
    },
    {
        "title": "Scene 2",
        "image": "output/scene2.png",
        "narration": "Priya explains recycling.",
        "dialogue": "Priya: Let's separate the waste."
    }
]
if st.button("📖 Create Comic PDF"):

    pdf_file = create_comic_pdf(
        panels,
        "output/my_comic.pdf"
    )

    st.success("Comic PDF created successfully! 🎉")

    with open(pdf_file, "rb") as file:

        st.download_button(
            label="⬇️ Download Comic PDF",
            data=file,
            file_name="AI_Comic_Story.pdf",
            mime="application/pdf"
        )
```

```
User
  ↓
📚 Topic
  ↓
🤖 Gemini
  ↓
✍️ Story + Dialogue
  ↓
🎬 Scenes
  ↓
🖼️ AI Images
  ↓
📖 Comic Panels
  ↓
📄 PDF
```

```
AI_Comic_Story_Creator/
│
├── app.py
├── gemini_api.py
├── image_generator.py
├── pdf_generator.py
├── requirements.txt
└── output/
```

```
streamlit
google-genai
pillow
reportlab
```

```
from google import genai

client = genai.Client(api_key="YOUR_GEMINI_API_KEY")


def generate_comic_story(topic, characters, language, scenes):

    prompt = f"""
    Create an educational comic story.

    Topic: {topic}
    Characters: {characters}
    Language: {language}
    Number of scenes: {scenes}

    For each scene provide:

    SCENE:
    Scene description

    DIALOGUE:
    Character and dialogue

    NARRATION:
    Short narration

    IMAGE PROMPT:
    Detailed prompt for creating the comic image.

    Make the story simple, creative and suitable for students.
    """

    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=prompt
    )
```

```
    return response.text
import streamlit as st
from gemini_api import generate_comic_story

st.set_page_config(
    page_title="AI Comic Story Creator",
    page_icon="🎨"
)

st.title("🎨 AI Comic Story Creator")
st.write("Create educational comic stories using Gemini AI")

topic = st.text_input(
    "📚 Enter Story Topic"
)

characters = st.text_input(
    "👥 Enter Character Names"
)

language = st.selectbox(
    "🌐 Select Language",
    ["Tamil", "English"]
)

scenes = st.number_input(
    "🎬 Number of Scenes",
    min_value=2,
    max_value=10,
    value=6
)

if st.button("✨ Generate Comic Story"):

    if topic and characters:

        with st.spinner(
            "🤖 Gemini is creating your story..."
        ):

            story = generate_comic_story(
                topic,
                characters,
                language,
                scenes
            )
```

```
        st.success("Comic story created! 🎉")

        st.subheader("📖 Your Comic Story")

        st.write(story)

    else:

        st.warning(
            "Please enter topic and character names."
        )
```

🎨 AI Comic Story Creator

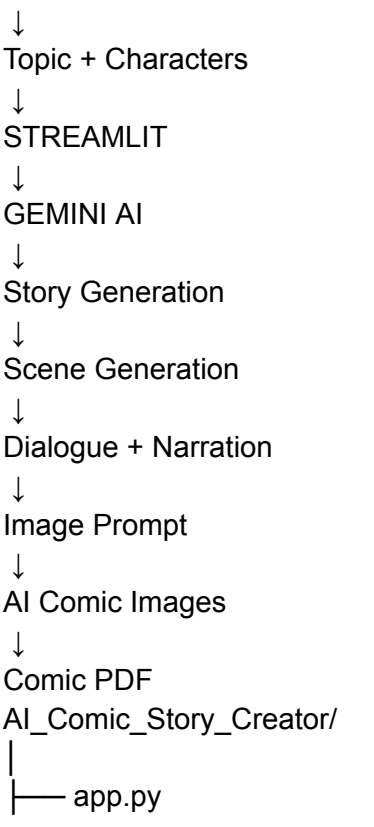
📚 Enter Story Topic
[Save Environment]

👥 Enter Character Names
[Arun, Priya]

🌐 Select Language
[Tamil]

🎬 Number of Scenes
[6]

[✨ Generate Comic Story]




```
AI_Comic_Story_Creator/
│
├── app.py
```

```
├── gemini_api.py
├── image_generator.py
├── pdf_generator.py
├── requirements.txt
└── output/
```

```
import streamlit as st
from gemini_api import generate_comic_story

st.set_page_config(
    page_title="AI Comic Story Creator",
    page_icon="🎨",
    layout="wide"
)

# ----------------------------
# Sidebar
# ----------------------------

st.sidebar.title("🎨 AI Comic Creator")

page = st.sidebar.radio(
    "Navigation",
    ["🏠 Home", "📖 Create Comic", "ℹ️ About"]
)

# ----------------------------
# HOME
# ----------------------------

if page == "🏠 Home":

    st.title("🎨 AI Comic Story Creator")

    st.subheader(
        "Create Educational Comics Using Gemini AI"
    )

    st.write(
        """
        This application converts your story idea into
        an educational comic story using Artificial Intelligence.
        """
    )

    st.info(
        "Go to 'Create Comic' from the sidebar to start."
    )
```

```
# ----------------------------
# CREATE COMIC
# ----------------------------

elif page == "📖 Create Comic":

    st.title("📖 Create Your Comic")

    topic = st.text_input(
        "📚 Story Topic",
        placeholder="Example: Save Environment"
    )

    characters = st.text_input(
        "👥 Characters",
        placeholder="Example: Arun, Priya"
    )

    language = st.selectbox(
        "🌐 Language",
        ["Tamil", "English"]
    )

    scenes = st.slider(
        "🎬 Number of Scenes",
        2,
        10,
        6
    )

    if st.button("✨ Generate Comic"):

        if topic and characters:

            with st.spinner(
                "🤖 Gemini is creating your comic..."
            ):

                story = generate_comic_story(
                    topic,
                    characters,
                    language,
                    scenes
                )

            st.success(
                "Comic story generated successfully! 🎉"
```

```
            )

            st.subheader("📖 Generated Story")

            st.write(story)

        else:

            st.warning(
                "Please enter both topic and characters."
            )

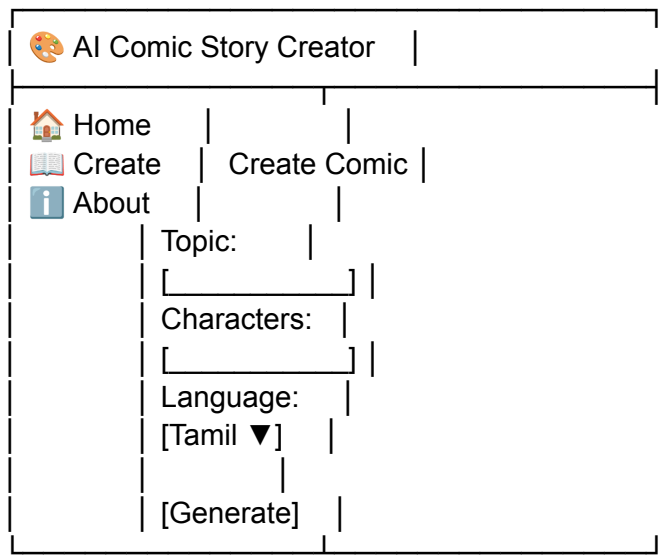

# ----------------------------
# ABOUT
# ----------------------------

else:

    st.title("ℹ️ About the Project")

    st.write(
        """
        AI Comic Story Creator is an educational AI project
        that uses the Gemini model to generate stories,
        scenes, dialogues and image prompts.

        Developed as part of the Naan Mudhalvan project.
        """
    )
```




```
from google import genai
import json
```

```
client = genai.Client(api_key="YOUR_GEMINI_API_KEY")


def generate_comic_story(topic, characters, language, scenes):

    prompt = f"""
    Create an educational comic story.

    Topic: {topic}
    Characters: {characters}
    Language: {language}
    Number of scenes: {scenes}

    Return ONLY valid JSON.

    Format:
    {{
        "title": "Comic title",
        "scenes": [
            {{
                "scene_number": 1,
                "description": "Scene description",
                "narration": "Short narration",
                "dialogue": [
                    {{
                        "character": "Character name",
                        "text": "Dialogue"
                    }}
                ],
                "image_prompt": "Detailed image prompt"
            }}
        ]
    }}

    Make the story educational and suitable for students.
    """

    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=prompt
    )

    text = response.text.strip()

    # Remove markdown code fences if Gemini adds them
    if text.startswith("```"):
        text = text.replace("```json", "")
        text = text.replace("```", "")
```

```
        text = text.strip()

    return json.loads(text)
comic = generate_comic_story(
    topic,
    characters,
    language,
    scenes
)

st.success("Comic created successfully! 🎉")

st.title("📖 " + comic["title"])

for scene in comic["scenes"]:

    st.divider()

    st.header(
        f"🎬 Scene {scene['scene_number']}"
    )

    st.write(
        "📍 " + scene["description"]
    )

    st.info(
        "📖 " + scene["narration"]
    )

    st.subheader("💬 Dialogue")

    for dialogue in scene["dialogue"]:

        st.write(
            f"**{dialogue['character']}:** "
            f"{dialogue['text']}"
        )

    st.caption(
        "🖼️ Image Prompt: "
        + scene["image_prompt"]
    )
```

🎬 Scene 1

📍 Arun sees plastic waste near his school.

📖 Arun decides to clean the area.

💬 Dialogue

Arun:
"நாம் சுற்றுச்சூழலை பாதுகாக்க வேண்டும்."

🖼️ Image Prompt:
Two school students near a school...
———————————————————

🎬 Scene 2

📍 Priya explains recycling.

📖 They separate plastic and paper waste.

💬 Dialogue

Priya:
"குப்பைகளை தனித்தனியாக பிரிப்போம்."

———————————————————

🎬 Scene 3

```
from google import genai

client = genai.Client(api_key="YOUR_GEMINI_API_KEY")


def generate_image(prompt, output_file):

    response = client.models.generate_content(
        model="gemini-2.5-flash-image",
        contents=prompt
    )

    for part in response.parts:

        if part.inline_data is not None:

            image = part.as_image()
            image.save(output_file)

            return output_file
```

```
    return None
os.makedirs("output", exist_ok=True)

for scene in comic["scenes"]:

    st.divider()

    st.header(
        f"🎬 Scene {scene['scene_number']}"
    )

    st.write(scene["description"])

    st.info(scene["narration"])

    st.subheader("💬 Dialogue")

    for dialogue in scene["dialogue"]:

        st.write(
            f"**{dialogue['character']}:** "
            f"{dialogue['text']}"
        )

    if st.button(
        f"🖼️ Generate Scene {scene['scene_number']} Image",
        key=f"image_{scene['scene_number']}"
    ):

        image_path = (
            f"output/scene_{scene['scene_number']}.png"
        )

        with st.spinner("Creating comic image..."):

            result = generate_image(
                scene["image_prompt"],
                image_path
            )

        if result:

            st.image(
                result,
                caption=f"Scene {scene['scene_number']}"
            )
```

```
            st.success("Image created! 🎉")

        else:

            st.error(
                "Could not generate image."
            )
from reportlab.pdfgen import canvas
from reportlab.lib.pagesizes import A4
from reportlab.lib.utils import ImageReader


def create_pdf(images, output_file):

    width, height = A4

    pdf = canvas.Canvas(
        output_file,
        pagesize=A4
    )

    for image_path in images:

        pdf.drawImage(
            ImageReader(image_path),
            40,
            250,
            width=515,
            height=350,
            preserveAspectRatio=True
        )

        pdf.showPage()

    pdf.save()

    return output_file
images = []

for i in range(1, scenes + 1):

    path = f"output/scene_{i}.png"

    if os.path.exists(path):
        images.append(path)
if images:

    if st.button("📄 Create Comic PDF"):
```

```
pdf_path = create_pdf(
    images,
    "output/AI_Comic_Story.pdf"
)

st.success(
    "Your comic PDF is ready! 🎉"
)

with open(pdf_path, "rb") as file:

    st.download_button(
        "⬇️ Download Comic PDF",
        file,
        file_name="AI_Comic_Story.pdf",
        mime="application/pdf"
    )
```